

Tối ưu hóa cắt tỉa trong phương pháp tìm kiếm

Qi Xin

Nguồn gốc: Pruning optimization in search methods

IOI China candidate team papers / 1999 / Qi Xin.pdf

Từ khóa. Tìm kiếm; tối ưu hóa; cắt tỉa.

Tóm tắt

Bài viết này thảo luận một kỹ thuật tối ưu hóa thường gặp nhất trong phương pháp tìm kiếm: cắt tỉa. Trọng tâm chính là thiết kế tiêu chuẩn phán đoán để cắt tỉa. Trước hết, bài viết dùng cây tìm kiếm để giải thích một cách trực quan cắt tỉa là gì; sau đó phân tích ba nguyên tắc khi thiết kế tiêu chuẩn cắt tỉa: đúng đắn, chính xác và hiệu quả. Bài viết chia các hướng thiết kế tiêu chuẩn cắt tỉa thường gặp thành hai loại lớn, cắt tỉa theo tính khả thi và cắt tỉa theo tính tối ưu, rồi dùng mỗi loại một bài thi làm ví dụ để minh họa ba nguyên tắc trên. Cuối cùng, bài viết tổng kết một số nhận xét về phương pháp cắt tỉa.

1 Dẫn nhập

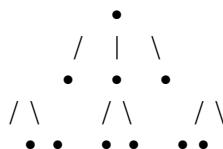
Tìm kiếm là một phương pháp cơ bản trong trí tuệ nhân tạo, đồng thời cũng là một phương pháp mà thí sinh thi tin học phải nắm vững. Khi xây dựng một thuật toán tìm kiếm, hai vấn đề đầu tiên thường không nằm ngoài:

1. Xây dựng cấu trúc thuật toán.
2. Chọn cấu trúc dữ liệu thích hợp.

Tuy nhiên, như mọi người đều biết, độ phức tạp thời gian của phần lớn phương pháp tìm kiếm là cấp mũ. Một tìm kiếm đơn giản không tối ưu hóa thường có hiệu suất thời gian thấp đến mức khó chấp nhận, càng khó đáp ứng giới hạn thời gian nghiêm ngặt trong thi tin học.

Nội dung chính của bài viết này là một phương pháp cơ bản để tối ưu hóa chương trình sau khi cấu trúc thuật toán đã được xây dựng: cắt tỉa.

Trước hết cần làm rõ ý nghĩa của “cắt tỉa”. Ta biết rằng quá trình tìm kiếm có thể xem như quá trình xuất phát từ gốc cây và duyệt một cây lộn ngược, tức cây tìm kiếm. Gọi là cắt tỉa vì thông qua một tiêu chuẩn phán đoán nào đó, ta tránh được một số quá trình duyệt không cần thiết. Nói hình ảnh hơn, ta cắt bỏ một số “cành” trong cây tìm kiếm, nên gọi là cắt tỉa.



Hình 1. Cây tìm kiếm, trong đó một số nhánh có thể bị loại bỏ trước khi duyệt.

Khi viết chương trình tìm kiếm, nói chung ta đều phải xét đến cắt tỉa. Hiển nhiên, vấn đề cốt lõi của tối ưu hóa bằng cắt tỉa là thiết kế tiêu chuẩn phán đoán: phương pháp xác định nhánh nào nên bỏ, nhánh nào nên giữ. Một tiêu chuẩn tốt thường có thể rút ngắn đáng kể thời gian chạy; ngược lại, nó cũng có thể gây phản tác dụng. Vì vậy trước hết ta cần phân tích các nguyên tắc cần tuân theo khi thiết kế tiêu chuẩn cắt tỉa.

2 Các nguyên tắc cắt tỉa

Nguyên tắc thứ nhất: đúng đắn. Như đã nói ở trên, cắt tỉa có thể tối ưu hóa hiệu suất thực thi vì nó có thể “cắt bỏ” một số “cành” của cây tìm kiếm. Nhưng nếu khi cắt tỉa ta cắt luôn nhánh “mọc ra” lời giải cần tìm, mọi tối ưu hóa đều mất ý nghĩa. Vì vậy yêu cầu đầu tiên đối với cắt tỉa là tính đúng đắn: phải bảo đảm không đánh mất kết quả đúng. Đây là tiền đề của tối ưu hóa bằng cắt tỉa.

Để thỏa mãn nguyên tắc này, ta nên dùng **điều kiện cần** để phán đoán cắt tỉa. Nói cách khác, ta xem nhánh đang xét có thể bị cắt hay không thông qua các đặc trưng mà lời giải bắt buộc phải có, các điều kiện mà lời giải bắt buộc phải thỏa, v.v. Như vậy có thể bảo đảm rằng nhánh bị cắt chắc chắn không phải nhánh chứa lời giải đúng. Tất nhiên, theo định nghĩa điều kiện cần, không bị cắt không có nghĩa là nhất định sẽ thu được lời giải đúng, nếu không thì cũng chẳng cần tìm kiếm nữa.

Nguyên tắc thứ hai: chính xác. Trên cơ sở bảo đảm đúng đắn, yêu cầu thứ hai đối với tiêu chuẩn cắt tỉa là tính chính xác, tức là có thể cắt càng nhiều nhánh không dẫn đến lời giải đúng càng tốt. Chỉ khi có độ chính xác cao, cắt tỉa mới thật sự phát huy hiệu quả tối ưu hóa. Vì thế có thể nói chính xác là sinh mệnh của tối ưu hóa bằng cắt tỉa.

Dĩ nhiên, để nâng cao độ chính xác của tiêu chuẩn cắt tỉa, ta phải phân tích đặc điểm của bài toán một cách toàn diện và tỉ mỉ, cố gắng phát hiện bản chất của bài toán để thiết kế được phương án phán đoán tốt.

Nguyên tắc thứ ba: hiệu quả. Nói chung, sau khi thiết kế tiêu chuẩn cắt tỉa, ta phải thực hiện một phép phán đoán trên mỗi nhánh của cây tìm kiếm. Nhưng vì phép phán đoán dựa trên “điều kiện cần” để có lời giải, chắc chắn vẫn có nhiều nhánh không chứa lời giải đúng mà không bị cắt. Trong các trường hợp ấy, thao tác phán đoán cắt tỉa rõ ràng có tác dụng phụ đối với hiệu suất chương trình. Để giảm tác dụng phụ này, ngoài việc cải thiện độ chính xác của phán đoán, ta thường còn phải nâng cao hiệu suất thời gian của chính thao tác phán đoán.

Điều này dẫn đến một mâu thuẫn: để tăng hiệu quả tối ưu hóa, ta phải nâng cao độ chính xác của tiêu chuẩn cắt tỉa, nên thường buộc phải tăng độ phức tạp của thao tác phán đoán, đồng thời làm giảm hiệu suất thời gian của nó. Nhưng nếu thời gian tiêu tốn cho phán đoán quá lớn, nó có thể làm giảm, thậm chí triệt tiêu hoàn toàn hiệu quả tối ưu hóa do độ chính xác cao hơn mang lại. Trong nhiều trường hợp, việc giải quyết tốt mâu thuẫn này thường trở thành điểm then chốt của tối ưu hóa thuật toán tìm kiếm.

Tóm lại, các nguyên tắc chính của tối ưu hóa bằng cắt tỉa có thể quy về ba từ: đúng, chính xác, hiệu quả.

Khi áp dụng cắt tỉa, chỉ có các nguyên tắc trên vẫn chưa đủ; ta còn cần nghiên cứu cụ thể các hướng thiết kế tiêu chuẩn cắt tỉa. Có thể chia đại khái các phán đoán cắt tỉa thường dùng thành hai loại:

1. Cắt tỉa theo tính khả thi.
2. Cắt tỉa theo tính tối ưu, hay cắt tỉa bằng cận trên và cận dưới.

Sau đây ta sẽ kết hợp ba nguyên tắc trên để thảo luận từng loại.

3 Cắt tỉa theo tính khả thi

Ta đã biết quá trình tìm kiếm có thể xem như việc duyệt một cây. Trong rất nhiều trường hợp, không phải mọi nhánh của cây tìm kiếm đều có thể dẫn đến kết quả ta cần; nhiều nhánh thực chất chỉ là ngõ cụt. Nếu ngay khi vừa bước vào một ngõ cụt như vậy ta đã nhận ra và lập tức cắt tỉa, hiệu suất chương trình thường sẽ được cải thiện. Cắt tỉa theo tính khả thi chính là xuất phát từ suy nghĩ này.

Hãy xét một ví dụ: “Betsy đi du lịch” của USACO.

Tóm tắt đề bài. Một thị trấn hình vuông được chia thành N^2 ô vuông nhỏ. Betsy cần đi từ ô góc trái trên đến ô góc trái dưới, và đi qua mỗi ô đúng một lần. Với N cho trước, hãy tính số tuyến đường mà Betsy có thể đi.

Ta dùng phương pháp quay lui theo chiều sâu để giải bài này. Betsy xuất phát từ góc trái trên; mỗi bước có thể di chuyển từ một ô sang một ô kề cạnh chưa đi qua. Gặp ngõ cụt thì quay lui. Khi đã đi $N^2 - 1$ bước và đến góc trái dưới, ta thu được một đường mới, rồi tiếp tục quay lui tìm kiếm cho đến khi duyệt hết mọi đường.

Tuy nhiên, nếu chỉ lập trình theo khung thuật toán trên, hiệu suất thời gian rất thấp; trường hợp $N = 6$ cũng không được xử lý tốt. Vì vậy tối ưu hóa là điều bắt buộc. Điểm then chốt của tối ưu hóa bài này là ở một bước tìm kiếm nào đó, có thể phán đoán trước liệu trong phần tìm kiếm phía sau chắc chắn sẽ gặp ngõ cụt hay không. Cắt tỉa theo tính khả thi có thể phát huy tác dụng tại đây.

Trước hết ta phân tích phương pháp cắt tỉa từ góc độ “điều kiện cần”, tức các đặc trưng mà một lời giải hợp lệ phải có. Có hai hướng chính:

1. Với một đường đi hợp lệ, ngoài điểm xuất phát và ô đích, mỗi ô trung gian tất yếu có quá trình “vào một lần, ra một lần”. Vì vậy trong quá trình tìm kiếm phải bảo đảm mỗi ô chưa đi qua còn kề với ít nhất hai ô chưa đi qua, trừ khi lúc ấy Betsy đang đứng ngay bên cạnh nó. Đây là phân tích ở góc độ vi mô.
2. Trên cơ sở điều kiện thứ nhất, ta còn có thể phân tích từ góc độ vĩ mô để nâng cao thêm độ chính xác của cắt tỉa. Rõ ràng tại mọi thời điểm của một phương án di chuyển hợp lệ, không thể tồn tại vùng bị cô lập. Dù mỗi ô trong vùng cô lập vẫn có thể có ít nhất hai ô trống kề cạnh, xét cả vùng như một chỉnh thể thì Betsy đã không thể đi tới đó nữa. Ta cũng nên kịp thời nhận ra và tránh trường hợp này.

Hai điều kiện cắt tỉa trên đều đúng và có độ chính xác khá cao. Nhưng chỉ thỏa hai điểm ấy vẫn chưa đủ; quá trình phán đoán cắt tỉa còn phải cố gắng hiệu quả. Nếu mỗi lần phán đoán ta đơn giản quét toàn bộ N^2 ô, hiệu suất thấp là điều dễ thấy. Do đó phải đơn giản hóa quá trình phán đoán hết mức có thể.

Thực ra, mỗi lần Betsy di chuyển chỉ ảnh hưởng đến các ô gần đó. Vì vậy mỗi lần thực hiện phán đoán cắt tỉa, ta chỉ nên kiểm tra các ô lân cận cô ấy.

Với điều kiện cắt tỉa thứ nhất, có thể đặt một mảng đánh dấu kiểu nguyên, lưu riêng cho mỗi ô số ô kề cạnh chưa được đi qua. Mỗi lần Betsy đi đến vị trí mới, nhiều nhất chỉ có bốn ô kề cạnh vị trí ấy bị thay đổi giá trị đánh dấu, nên chỉ cần kiểm tra các giá trị này.

Với điều kiện cắt tỉa thứ hai, xử lý hơi phức tạp hơn. Nhưng ta vẫn có thể dùng phương pháp phân tích cục bộ: chỉ dựa vào các ô gần Betsy để xác định có nên cắt tỉa hay không. Nguyên lý có thể mô tả như sau. Ký hiệu X là ô đã đi qua, O là ô chưa đi qua, B là vị trí mới của Betsy, và mũi tên là hướng di chuyển:

O	X	O	X	X	O	O	X	X
O	B↑	O	O	B↑	O	O	B↑	O

Ba cấu hình trên là ba tình huống có thể cắt tỉa. Vì mọi ô Betsy đã đi qua chắc chắn tạo thành một vùng

liên thông bốn hướng, trong mỗi tình huống hai ô trắng được tách bởi vệt đã đi qua chắc chắn không liên thông với nhau. Ít nhất một trong hai phía thuộc về một vùng cô lập, nên đều có thể cắt tĩa.

Sau các tối ưu hóa trên, hiệu suất thời gian của chương trình được cải thiện rất nhiều; xem phụ lục.

Nói chung, cắt tĩa theo tính khả thi thường dùng trong các bài tìm kiếm đường đi. Ngoài ví dụ này, các bài như “Prime Circle” ở ACM Asian Regional 1996 cũng có thể dùng phương pháp cắt tĩa này.

Khi áp dụng cắt tĩa theo tính khả thi, trước hết cần phân tích đặc điểm bài toán từ nhiều góc độ, tìm càng nhiều tình huống có thể cắt tĩa càng tốt. Đồng thời, cũng phải chú ý nâng cao hiệu suất thời gian của cắt tĩa, nên ở đây ta dùng phương pháp “phán đoán cục bộ”. Đặc biệt khi xử lý điều kiện cắt tĩa thứ hai, ta dùng phán đoán cục bộ để thể hiện tính chất toàn cục, tức có tồn tại vùng cô lập hay không. Kỹ thuật này không chỉ có ích khi thiết kế phương pháp cắt tĩa, mà còn được ứng dụng rất rộng rãi ở nhiều mặt khác.

4 Cắt tĩa theo tính tối ưu

Trong các bài ta thường gặp, có một lớp lớn gọi là bài toán tối ưu hóa, tức kết quả cần tìm là lời giải tối ưu. Nếu dùng tìm kiếm để giải các bài như vậy, cắt tĩa theo tính tối ưu là điều chắc chắn phải xét đến.

Để thống nhất cách trình bày, trước hết cần nói rõ một điểm. Ta biết rằng độ tốt xấu của lời giải thường được đánh giá bằng một hàm đánh giá. Ở đây định nghĩa một hàm đánh giá trừu tượng gọi là “độ ưu”, giá trị càng lớn thì lời giải tương ứng càng tốt. Với bài cụ thể, có thể xem “độ ưu” là lợi ích dương, hoặc là chi phí lấy dấu âm, v.v.

Sau đó, hãy nhìn lại quá trình tìm kiếm lời giải tối ưu. Nói chung, ta cần lưu một “lời giải tốt nhất hiện tại”, thực chất là lưu một cận dưới của độ ưu. Khi duyệt đến nút lá của cây tìm kiếm, ta thu được một lời giải mới và do đó cũng thu được giá trị hàm đánh giá của nó. So sánh giá trị ấy với cận dưới đang lưu; nếu độ ưu của lời giải mới lớn hơn, nó trở thành cận dưới mới. Sau khi tìm kiếm kết thúc, lời giải đang lưu là lời giải tối ưu.

Vậy cắt tĩa theo tính tối ưu tiến hành như thế nào? Khi đang ở trên một nhánh của cây tìm kiếm, ta có thể dùng một phương pháp nào đó để ước lượng cận trên của hàm đánh giá đối với mọi lời giải trên nhánh ấy, tức hàm ước lượng h . Hiển nhiên, h lớn hơn cận dưới độ ưu hiện tại là điều kiện cần để trên nhánh ấy tồn tại lời giải tối ưu; nếu không, chắc chắn có thể cắt tĩa. Vì vậy cắt tĩa theo tính tối ưu cũng có thể gọi là cắt tĩa bằng cận trên và cận dưới. Đồng thời ta cũng thấy vấn đề cốt lõi của loại cắt tĩa này là xây dựng hàm ước lượng.

Sau đây là một ví dụ điển hình: “số lần nhân ít nhất”.

Tóm tắt đề bài. Bắt đầu từ x , hãy dùng số phép nhân ít nhất để thu được x^n , trong đó n là dữ liệu vào.

Vì nhân hai lũy thừa tương đương cộng hai số mũ, bài này có thể biểu diễn tương đương như sau. Cần xây dựng một dãy $\{a_i\}$ thỏa

$$a_i = \begin{cases} 1, & i = 1, \\ a_j + a_k, & 1 \leq j, k < i, i > 1, \end{cases}$$

yêu cầu $a_t = n$ và làm cho t nhỏ nhất.

Ta chọn quay lui làm cấu trúc chính. Khi tìm kiếm đến tầng i , giá trị a_i nằm trong khoảng từ $a_{i-1} + 1$ đến $2a_{i-1}$. Để nhanh chóng tiến gần mục tiêu n , nên thử giá trị cho a_i từ $2a_{i-1}$ giảm dần, dĩ nhiên các giá trị được chọn không được vượt quá n . Khi tìm thấy n , ta thu được một lời giải và so sánh với lời giải đang lưu để lấy tốt hơn. Khi tìm kiếm kết thúc, ta thu được lời giải tối ưu cuối cùng.

Nếu trực tiếp tìm kiếm theo cấu trúc trên, hiệu suất rõ ràng rất thấp. Vì vậy ta có thể dùng cắt tĩa theo tính tối ưu để tối ưu hóa.

Tối ưu thứ nhất. Trước hết phải xây dựng hàm ước lượng. Vì nhân đôi số lớn nhất trong dãy chắc chắn là cách tăng nhanh nhất, một hàm ước lượng đơn giản nhất là, giả sử số lớn nhất hiện tại trong dãy là a_i , tức độ sâu tìm kiếm hiện tại là i :

$$h = \left\lceil \log_2 \frac{n}{a_i} \right\rceil.$$

Tuy nhiên, độ chính xác của hàm ước lượng này quá kém. Đặc biệt khi a_i lớn hơn $n/2$, h chỉ bằng 1, gần như không thể phát huy tác dụng cắt tỉa. Vì vậy, dù dùng quay lui theo chiều sâu hay tìm kiếm A^* theo chiều rộng, nếu vẫn dùng hàm ước lượng này thì hiệu quả tối ưu hóa đều khá hạn chế.

Tối ưu thứ hai. Hướng vào “sinh mệnh” của hàm ước lượng, tức độ chính xác, ta có thể dùng một mở rộng của tính chất cộng theo chuỗi để cải thiện độ chính xác của hàm ước lượng trong một số trường hợp. Chi tiết cải tiến được nêu trong phụ lục.

Tối ưu thứ ba. Hàm ước lượng mới có độ chính xác cao hơn một chút, nhưng độ phức tạp thời gian cũng tăng tương ứng. Để nâng cao hiệu suất cắt tỉa, ta có thể dùng kỹ thuật “tinh chỉnh từng bước”: trước hết dùng hàm ước lượng cũ, nhanh nhưng kém chính xác, để lọc một lần; sau đó dùng hàm ước lượng mới, chính xác hơn một chút nhưng chậm hơn, để giảm số trường hợp lọt qua, nhằm cân bằng giữa độ chính xác và hiệu suất chạy.

Tối ưu thứ tư. Trước khi tìm kiếm, ta có thể phân tích n thành thừa số nguyên tố, rồi dùng phương pháp tìm kiếm nói trên giải riêng cho từng thừa số nguyên tố. Nếu một thừa số nguyên tố vẫn quá lớn, còn có thể xét nó trừ đi 1 rồi phân tích tiếp. Điều này tương đương chia quá trình xây dựng dãy thành nhiều giai đoạn để xử lý. Kết quả thu được tuy chưa chắc là tối ưu toàn cục, nhưng có thể dùng làm cận dưới ban đầu của độ ưu, giúp quá trình tìm kiếm toàn cục phía sau tránh rất nhiều đường vòng. Thực tế, phương pháp ước lượng này khá chính xác; trong nhiều trường hợp nó trực tiếp cho lời giải tối ưu, nhờ đó hiệu suất chương trình tăng rất mạnh.

Tối ưu thứ năm. Khi số lớn nhất hiện tại a_i trong dãy vượt quá $n/2$, hàm ước lượng cũ khó phát huy tác dụng. Nhưng phương án tối ưu sau đó thực chất tương đương với việc chọn càng ít số càng tốt từ i số $a_1, \dots, a_i$, cho phép lặp lại, sao cho tổng của chúng bằng n . Bài toán này đã có thể giải bằng quy hoạch động. “Hàm ước lượng” thu được theo cách này không chỉ hoàn toàn chính xác, mà thậm chí còn có thể thay thế trực tiếp quá trình tìm kiếm phía sau. Ở đây cũng thể hiện sự bổ sung ưu thế giữa tìm kiếm và quy hoạch động.

Bài này chứa nhiều kỹ thuật cắt tỉa theo tính tối ưu, và hiệu quả tối ưu hóa của chúng cũng rất rõ rệt; xem phân tích trong phụ lục.

Từ bài này, kết hợp với kinh nghiệm trước đây, ta có thể rút ra một số điểm cần chú ý khi áp dụng cắt tỉa theo tính tối ưu:

1. Thiết kế hàm ước lượng trước hết tất nhiên phải thỏa nguyên tắc đúng đắn, tức dùng “điều kiện cần” để cắt tỉa. Trên cơ sở đó cần chú ý nâng cao độ chính xác của ước lượng. Tối ưu thứ hai của bài này nhằm mục đích ấy.
2. Cũng như các thao tác phán đoán cắt tỉa khác, khi tăng độ chính xác của hàm ước lượng trong cắt tỉa theo tính tối ưu, đồng thời phải chú ý làm cho quá trình tính toán hiệu quả nhất có thể. Vì phán đoán cắt tỉa được thực hiện cực kỳ thường xuyên khi chạy, việc mài giũa thuật toán của nó là rất cần thiết. Đôi khi còn có thể dùng một số thủ pháp đặc biệt, như kỹ thuật “tinh chỉnh từng bước” ở trên, để tìm điểm cân bằng giữa hiệu suất thời gian và độ chính xác.

3. Khi dùng cắt tỉa theo tính tối ưu, một cận dưới “độ ưu” ban đầu tốt thường rất quan trọng. Trước khi bắt đầu tìm kiếm, ta có thể dùng một phương pháp hiệu quả nào đó, chẳng hạn tham lam, để tìm một lời giải tương đối tốt làm cận dưới ban đầu; cách này thường cắt bỏ được rất nhiều nhánh rõ ràng không thể tốt. Bài này dùng tìm kiếm theo giai đoạn để định cận ban đầu, và nhờ đó đạt tối ưu hóa lớn.
4. Ví dụ trong mục này là việc áp dụng cắt tỉa theo tính tối ưu trong tìm kiếm theo chiều sâu. Dĩ nhiên, trong tìm kiếm theo chiều rộng cũng có thể dùng cắt tỉa theo tính tối ưu; đó chính là phương pháp nhánh cận thường nói. Bài này cũng có thể giải bằng cách kết hợp nhánh cận với thuật toán A^* , dùng hàm ước lượng cải tiến làm ước lượng cận trên độ ưu, tức hàm h^* , và dùng quy hoạch động nói trên làm ước lượng cận dưới độ ưu. Tuy vậy, hiệu suất thời gian và không gian đều kém hơn. Nhưng trong một số bài, sự kết hợp giữa nhánh cận và A^* , nhờ tính ổn định khá tốt, vẫn có đất dụng võ.

5 Tổng kết

Do phương pháp tìm kiếm có “khiếm khuyết bẩm sinh” về hiệu suất thời gian, người ta đã nghiên cứu nhiều kỹ thuật tối ưu hóa nhằm vào nó. “Cắt tỉa” mà bài viết này bàn đến là một trong các phương pháp tối ưu hóa thường gặp nhất; có thể nói hầu như chỉ cần muốn dùng thuật toán tìm kiếm để giải bài, ta đều phải xét đến tối ưu hóa bằng cắt tỉa.

Ngoài ra cần nói rõ rằng hai loại phán đoán cắt tỉa được giới thiệu trong bài này, tính khả thi và tính tối ưu, chỉ được phân loại theo đối tượng ứng dụng khác nhau. Ba nguyên tắc đã trình bày ở trên, đúng đắn, chính xác và hiệu quả, mới là linh hồn xuyên suốt từ đầu đến cuối.

Bài viết còn giới thiệu một số kỹ thuật thường dùng trong cắt tỉa, như “phân tích cục bộ”, “tinh chỉnh từng bước”, v.v. Dùng chúng thích hợp cũng có thể nâng cao đáng kể hiệu suất chương trình, chủ yếu là hiệu suất thời gian.

Nhưng dù phương pháp cắt tỉa tinh xảo đến đâu, nó cũng không thể hạ độ phức tạp thời gian của thuật toán tìm kiếm về bản chất; đây là sự thật không thể phủ nhận. Vì vậy, trước khi bắt tay thiết kế một thuật toán tìm kiếm, ta cũng nên cân nhắc xem có tồn tại phương pháp hiệu quả hơn hay không.

Hơn nữa, trong thi tin học còn có vấn đề độ phức tạp lập trình. Nếu ta dùng rất nhiều kỹ thuật tối ưu hóa cho một thuật toán tìm kiếm, tuy có thể cải thiện phần nào hiệu suất thời gian, nhưng thường phải tiêu tốn rất nhiều thời gian lập trình, rất dễ rơi vào kết quả “nhặt hạt vừng, đánh rơi quả dưa”.

Tóm lại, trong quá trình thiết kế chương trình thực tế, ta không nên chui vào ngõ hẹp, mà càng phải phát huy đầy đủ tính linh hoạt của tư duy, kiên trì phương pháp tư tưởng “phân tích cụ thể từng vấn đề cụ thể”.

Tài liệu tham khảo

1. Liu Fusheng, Wang Jiande, *Hướng dẫn Olympic Tin học quốc tế thanh thiếu niên: tìm kiếm trí tuệ nhân tạo và thiết kế chương trình*.
2. *Informatics Olympic*.

A Phụ lục

A.1 Tình hình kiểm thử của “Betsy đi du lịch”

Đơn vị thời gian là giây; môi trường kiểm thử là Pentium II 266 MHz, 64 MB.

N	Số đường	Không cắt	Cắt 1	Cắt 2	Cả hai
2	1	< 0.01	< 0.01	< 0.01	< 0.01
3	2	< 0.01	< 0.01	< 0.01	< 0.01
4	8	< 0.01	< 0.01	< 0.01	< 0.01
5	86	0.17	< 0.01	< 0.01	< 0.01
6	1770	41.25	0.44	0.33	0.11
7	88418	Rất lâu	28.06	26.86	8.51

Có thể thấy việc dùng các tiêu chuẩn cắt tĩa theo tính khả thi nói trên làm hiệu suất thời gian của chương trình tăng rất đáng kể.

A.2 Cải tiến hàm ước lượng của “số lần nhân ít nhất”

Ý tưởng thiết kế hàm ước lượng ban đầu thực chất là trên cơ sở dãy hiện tại $a_1, \dots, a_i$, lý tưởng hóa việc xây dựng

$$2a_i, 4a_i, \dots, 2^p a_i.$$

Khi $2^p a_i < n < 2^{p+1} a_i$, phương pháp ước lượng cũ cho rằng chỉ cần thêm một phép cộng nữa, tức tìm một số cộng với $2^p a_i$, là đủ.

Nhưng điều này chỉ bảo đảm có thể thu được một số lớn hơn hoặc bằng n , chưa chắc thu được đúng n . Ta phân tích như sau.

Trước hết, mọi số tự nhiên i đều có thể viết dưới dạng

$$2^k(2m+1) \quad (k, m \in \mathbb{Z}_{\geq 0}).$$

Đặt $k(i)$ là giá trị k tương ứng với số tự nhiên i . Rõ ràng với mọi hai số tự nhiên a, b , ta có

$$k(a+b) \geq \min\{k(a), k(b)\}.$$

Từ đây dễ liên tưởng đến các tính chất chặn lẻ như lẻ cộng lẻ bằng chẵn, chẵn cộng chẵn bằng chẵn, lẻ cộng chẵn bằng lẻ.

Xét dãy được xây dựng trong ước lượng ở trên:

$$a_1, a_2, \dots, a_i, 2a_i, 4a_i, \dots, 2^p a_i, \quad 2^p a_i < n < 2^{p+1} a_i.$$

Trước khi dùng phán đoán cắt tĩa mới, ta nên kiểm tra

$$\left\lceil \log_2 \frac{n}{a_i + a_{i-1}} \right\rceil \geq p,$$

vì điều kiện này bảo đảm rằng chỉ có xây dựng dãy trên mới là tối ưu.

Nếu tồn tại số tự nhiên j , $1 \leq j \leq p$, sao cho

$$k(2^j a_i) > k(n),$$

thì từ $k(a+b) \geq \min\{k(a), k(b)\}$, với $j \leq t \leq p$ ta có

$$k(2^t a_i + 2^p a_i) \geq k(2^t a_i) \geq k(2^j a_i) > k(n).$$

Do $k(a) = k(b)$ là điều kiện cần để $a = b$, suy ra

$$2^t a_i + 2^p a_i \neq n.$$

Nghĩa là mọi số trong $2^j a_i, \dots, 2^p a_i$ cộng với $2^p a_i$ đều không thể thu được n . Vì vậy, nếu

$$n - 2^p a_i > 2^{j-1} a_i,$$

thì sau khi thu được $2^p a_i$, ít nhất còn cần thêm hai phép cộng nữa mới có thể thu được n .

Cải tiến trên có thể làm hàm ước lượng chính xác hơn một chút trong một số trường hợp, nhưng bản thân nó có hiệu suất thời gian kém hơn. Lý do là hàm ước lượng mới không chỉ bao gồm hàm ước lượng cũ, tức quá trình xây dựng dãy, mà còn thêm các thao tác như tính $k(a_i)$. Vì vậy, dù hàm mới chỉ là một cải tiến của hàm cũ, không phải hai mặt bổ sung cho nhau như trong bài “Betsy đi du lịch”, trong chương trình thực tế vẫn cần gọi liên tiếp hai hàm ước lượng, tức dùng kỹ thuật “tinh chỉnh từng bước” đã nói ở trên.

Cuối cùng cần nói thêm rằng mọi phần dùng phép toán logarit trong các hàm ước lượng nói tới trong bài này đều phải đổi thành vòng lặp cộng dồn khi lập trình thực tế. Nguyên nhân là hàm số thực trong máy tính được tính bằng khai triển chuỗi, thực chất cũng là cấu trúc lặp, lại phức tạp và kém hiệu quả hơn. Dĩ nhiên, trong chương trình thực tế còn có một số tối ưu hóa chi tiết khác.

A.3 Kiểm thử và phân tích “số lần nhân ít nhất”

Một số dữ liệu kiểm thử, đơn vị giây. Môi trường kiểm thử là Pentium Pro 233 MHz, 64 MB.

N	Số nhân	$P1$	$P2$	$P3$	$P4$	$P5$	$P6$	$P7$
127	10	0.22	0.22	0.05	0.06	0.06	< 0.01	0.16
191	11	3.52	3.90	0.83	0.55	0.55	0.27	1.75
382	11	13.02	13.29	1.05	0.94	0.66	0.33	2.69
511	12	Rất lâu	Rất lâu	0.82	5.66	0.33	0.27	1.81
635	13	46.96	40.37	20.65	19.61	10.06	8.13	Tràn
719	13	45.59	39.11	45.64	13.45	39.11	6.64	27.35
1002	13	11.81	10.65	2.74	3.46	1.10	0.99	6.26
1023	13	Rất lâu	Rất lâu	2.66	Rất lâu	1.04	0.88	6.15
1357	13	9.51	6.98	5.93	5.44	3.40	2.74	13.35
1894	14	44.87	37.24	13.89	15.49	6.37	4.61	23.68

Khi $n \leq 1000$, thời gian chạy của chương trình 6 đều không vượt quá 10 giây, trong đó chỉ có hơn mười dữ liệu kiểm thử chạy trên 5 giây.

Chương trình	Cắt đơn giản	Cắt cải tiến	Định cận đầu	Quy hoạch động	A^* + nhánh cận
1	có				
2	có	có			
3	có		có		
4	có			có	
5	có	có	có		
6	có	có	có	có	
7					có

Trước hết so sánh hiệu quả tối ưu hóa khi ba điểm tối ưu chính được dùng riêng lẻ: phán đoán cắt tỉa cải tiến, tức tinh chỉnh từng bước; định cận ban đầu; và kết hợp quy hoạch động.

1. Chỉ thêm phương pháp cắt tỉa cải tiến, tức chương trình 2, không cải thiện nhiều hiệu suất thời gian. Lý do chính là độ chính xác của hàm ước lượng mới chỉ tăng nhẹ, trong khi hiệu suất thời gian của chính nó lại thấp, phần nào triệt tiêu lợi ích do ước lượng chính xác hơn đem lại.
2. Phương pháp định cận ban đầu của bài này, tức chương trình 3, có độ chính xác rất cao, thường trực tiếp thu được lời giải tối ưu, nên có thể cải thiện hiệu suất khá rõ rệt. Trong bốn chương trình đầu, chỉ nó hoàn thành mọi dữ liệu trong bảng trong vòng một phút, đặc biệt hiệu quả khi $n = 1023$. Nhưng nếu độ chính xác của định cận ban đầu giảm một chút, hiệu quả tối ưu hóa sẽ trở nên rất không rõ, như trường hợp $n = 719$.
3. Sau khi thêm quy hoạch động, chương trình 4 thực chất làm quá trình tìm kiếm chủ yếu tập trung dưới $\lfloor n/2 \rfloor$, tránh ở mức đáng kể việc tìm kiếm trong đoạn từ $\lfloor n/2 \rfloor$ đến n , nơi hàm ước lượng gần như không có tác dụng. Vì vậy chương trình 4 gần như tốt hơn chương trình 1 khá rõ ở mọi điểm kiểm thử.

Tiếp theo xét chương trình 5. Nhờ dùng định cận ban đầu, trước khi tìm kiếm đã có một cận dưới độ ưu tương đối chính xác, nên hàm ước lượng mới có độ chính xác cao hơn cũng có xác suất phát huy tác dụng cao hơn; do đó hiệu quả tối ưu hóa khá rõ. Cũng như chương trình 3, khi định cận ban đầu không thật chính xác, như $n = 719$, hiệu quả tối ưu hóa giảm mạnh.

Trong bảy chương trình, tốt nhất đương nhiên là chương trình 6. Vì tổng hợp nhiều phương pháp tối ưu hóa và để chúng bổ sung ưu thế cho nhau, nó đạt hiệu quả tối ưu hóa ổn định nhất.

Cuối cùng, ta còn thấy rằng trong trường hợp đặc thù của bài này, tìm kiếm quay lui theo chiều sâu sau khi đã tối ưu hóa đầy đủ rõ ràng tốt hơn thuật toán A^* cả về thời gian lẫn không gian. Chương trình 7 được biên dịch ở chế độ bảo vệ và ngoài A^* còn kết hợp nhánh cận; nếu không, hiệu suất thời gian và không gian còn kém hơn.

A.4 Phụ lục chương trình

Chương trình “Betsy đi du lịch” dùng hai kiểu cắt tỉa.

```
{ $A+, B-, D-, E+, F-, G+, I-, L-, N+, O-, P-, Q-, R-, S-, T-, V+, X+, Y- }
{ $M 65520, 0, 655360 }
program Betsy; {Vi du 1 cua bai viet doi tuyen tap huan IOI'99: Betsy di du lich}
{Ghi chu:
    1. De thuan tien do thoi gian, chuong trinh doc du lieu tu dong lenh.
    2. De xu ly tien hon, chuong trinh them mot vong o danh dau ngoai ban do.
    3. Cac doan tuong doi doc lap ve logic duoc tach bang dong trong.
}
const
    max=7; {Quy mo du lieu lon nhat ma chuong trinh nay xu ly duoc}
    delta=array[1..4, 1..2] of shortint=((-1,0),(0,1),(1,0),(0,-1)); {Do lech huong}

var
    map=array[0..max+1, 0..max+1] of integer;
    {Ban do dung de danh dau lo trinh cua Betsy:
        vi tri chua di qua duoc danh dau -1,
        vong o danh dau ben ngoai duoc danh dau 0,
        cac o khac ghi so buoc khi Betsy den o do
    }
    left=array[0..max+1, 0..max+1] of shortint;
    {Mang danh dau: ghi so o ke canh chua duoc Betsy di qua quanh moi o}
```

```

n:1..max;{Canh ban do}
n2:integer;{N*N}
s:longint;{Bo dem, ghi tong so lo trinh di chuyen cua Betsy}

procedure init;{Doc du lieu va khoi tao}
var
  i:integer;{Bien lap}
  temp:integer;{Bien tam}
begin
  val(paramstr(1),n,temp);{Doc n tu dong lenh}
  n2:=n*n;

  fillchar(map,sizeof(map),255);
  for i:=0 to n+1 do begin
    map[0,i]:=0;
    map[i,0]:=0;
    map[n+1,i]:=0;
    map[i,n+1]:=0;
  end;
  map[1,1]:=1;
  {Doan tren khoi tao ban do}

  fillchar(left,sizeof(left),4);
  for i:=2 to n-1 do begin
    left[1,i]:=3;left[n,i]:=3;
    left[i,1]:=3;left[i,n]:=3;
  end;
  left[1,1]:=2;left[1,n]:=2;left[n,1]:=3;left[n,n]:=2;
  dec(left[1,2]);dec(left[2,1]);
  {Doan tren khoi tao mang danh dau}

  s:=0;{Xoa bo dem}
end;

procedure change(x,y,dt:integer);{Cong dt vao cac gia tri left cua 4 o ke canh (x,y)}
{Khi dat danh dau thi dt=-1; khi quay lui thi dt=1}
var
  k:integer;{Bien lap}
  a,b:integer;{Toa do tam}
begin
  for k:=1 to 4 do begin
    a:=x+delta[k,1];b:=y+delta[k,2];
    inc(left[a,b],dt);
  end;
end;

procedure expand(step,x,y:integer);{Thu tuc tim kiem chinh: tim buoc step, mo rong tu (x,y)}
var

```

```

nx,ny:integer;{Vi tri moi cua Betsy}
dir:byte;{Huong di chuyen cua Betsy}

```

```

function able(x,y:integer):boolean;{Xet Betsy co the vao (x,y) hay khong}
begin
  able:=not((map[x,y]<>-1)or((step<>n2)and(x=n)and(y=1)));
end;

```

```

function cut1:boolean;{Phan doan cat tia 1}
var
  i:integer;{Bien lap}
  a,b:integer;{Toa do tam}
begin
  for i:=1 to 4 do begin
    a:=x+delta[i,1];b:=y+delta[i,2];
    if (map[a,b]=-1)and((a<>nx)or(b<>ny))and(left[a,b]<=1) then begin
      cut1:=true;
      exit;
    end;
  end;
  cut1:=false;
end;

```

```

function cut2:boolean;{Phan doan cat tia 2}
var
  d1,d2:integer;{Hai huong "trai" va "phai" so voi huong di hien tai}
  fx,fy:integer;{Vi tri neu Betsy di them mot buoc theo huong cu tu vi tri hien tai}
begin
  if (dir=2)or(dir=4) then begin
    d1:=1;d2:=3;
  end
  else begin
    d1:=2;d2:=4;
  end;
  fx:=nx+delta[dir,1];fy:=ny+delta[dir,2];

```

```

if (map[fx,fy]<>-1)and(map[nx+delta[d1,1],ny+delta[d1,2]]=-1)
and(map[nx+delta[d2,1],ny+delta[d2,2]]=-1) then begin
  cut2:=true;
  exit;
end;
if (map[fx+delta[d1,1],fy+delta[d1,2]]<>-1)and(map[nx+delta[d1,1],ny+delta[d1,2]]=-1)
and(map[fx,fy]=-1) then begin
  cut2:=true;
  exit;
end;
if (map[fx+delta[d2,1],fy+delta[d2,2]]<>-1)and(map[nx+delta[d2,1],ny+delta[d2,2]]=-1)
and(map[fx,fy]=-1) then begin
  cut2:=true;
  exit;
end;

```

```

end;
{Ba dieu kien tren tuong ung voi ba truong hop trong hinh Nguyen Ly Cat Tia}

cut2:=false;
end;

begin{Expand}
if (step>n2)and(x=n)and(y=1) then begin{Tim den tang day, tang bo dem len 1}
  inc(s);
  exit;
end;
for dir:=1 to 4 do begin{Thu lan luot 4 huong di chuyen}
  nx:=x+delta[dir,1];ny:=y+delta[dir,2];
  if able(nx,ny) then begin
    if cut1 then continue;{Goi phan doan cat tia 1}
    if cut2 then continue;{Goi phan doan cat tia 2}
    change(nx,ny,-1);
    map[nx,ny]:=step;
    expand(step+1,nx,ny);{Goi de quy tang tim kiem tiep theo}
    map[nx,ny]:=-1;
    change(nx,ny,1);
  end;
end;
end;

begin{Chuong trinh chinh}
  init;{Khoi tao}
  expand(2,1,1);{Goi qua trinh tim kiem}
  writeln('The number of tours is ',s);{Xuat ket qua}
end.

```

Chương trình “số lần nhân ít nhất”, tức chương trình 6 trong phụ lục.

```

{$A+,B-,D-,E+,F-,G+,I-,L-,N+,O-,P-,Q-,R-,S-,T-,V+,X+,Y-}
{$M 65520,0,655360}
program LeastMultiply;{Vi du 2 cua bai viet doi tuyen tap huan IOI'99: so lan nhan it nhat}
{Ghi chu:
  1. De thuan tien do thoi gian, chuong trinh doc n tu dong lenh.
  2. Sau khi ket thuc, chuong trinh ghi cach thuc hien phep nhan va tong so phep nhan vao output.txt.
  3. Trong qua trinh tim kiem, do co ket hop quy hoach dong, chuong trinh co the thu duoc do dai day loi giai toi uu khi chua tim den tang day. Luc do chua co day so mu toi uu day du, nen sau khi tim kiem ket thuc, chuong trinh goi formkeep de sinh day cau tao day du trong keep.
  4. Qua trinh tim kiem sinh day so mu toi uu, chua truc tiep dua ra cach thuc hien cac phep nhan, nen thu tuc output chuyen doi no khi xuat ket qua.
  5. De nang cao hieu suat thoi gian, chuong trinh co vai toi uu hoa chi tiet; xem chu thích trong code.
  6. Cac doan tuong doi doc lap ve logic duoc tach bang dong trong.
}

const
  max=20;{Do dai lon nhat cua day}

```

```

maxr=2000;{Do dai lon nhat cua mang dem quy hoach dong; n dau vao khong duoc vuot qua 2*maxr}
mp=100;{Can tren cua cac so co the xu ly truc tiep bang tim kiem khi tien xu ly uoc luong}
power2:array[0..12]of integer= (1,2,4,8,16,32,64,128,256,512,1024,2048,4096); {Luy thua cua 2}

```

type

```

atype=array[0..max]of integer;{Kieu mang ghi day so mu duoc cau tao; phan tu 0 ghi do dai day}

```

var

```

n:integer;{So dich doc vao}
time:array[0..maxr]of integer;
{Mang dem quy hoach dong: time[i] la so so hang toi thieu can dung de tao so i tren co so day hien tai}
range:integer;{Pham vi xu ly bang quy hoach dong: range=[n/2]}
a:atype;{Day ghi trong luc tim kiem}
kp:atype;{Mang tam ghi day ket qua khi tien xu ly uoc can}
keep:atype;{Mang ghi day ket qua toi uu}
best:integer;{Loi giai toi uu hien tai}
f:text;{Tep xuat}

```

procedure init;{Khoi tao}

```

var
    temp:integer;{Bien tam}
begin
    val(paramstr(1),n,temp);{Doc n tu dong lenh}
    keep[0]:=1;
    keep[1]:=1;
    best:=maxint;{Gia tri dau cua do dai day toi uu}
    assign(f,'output.txt');{Noi voi tep xuat}
end;

```

procedure search(n:integer;var best:integer;var keep:atype);{Qua trinh tim kiem chinh}

```

{Truoc khi tim kiem, muc tieu tim kiem la n;
    best luu can duoi do uu da co truoc khi tim kiem;
    keep luu day so mu toi uu ung voi can duoi do uu ban dau.

```

```

Sau khi tim kiem ket thuc, best la do dai day so mu toi uu duoc cau tao,
tuc so lan nhan it nhat cong 1; keep la day so mu duoc cau tao.
}

```

```

var
    kn:integer;{So lan xuat hien thua so 2 trong n}
    i:integer;{Bien lap}

```

function getk(num:integer):integer;{Tinh so lan xuat hien thua so 2 trong num, tuc k(num) trong bai}

```

var
    i:integer;{Bien lap}
begin
    i:=0;
    while not odd(num) do begin
        num:=num shr 1;
        inc(i);
    end;

```

```

    getk:=i-1;
end;

```

```

procedure find(step:integer);{Qua trình tìm kiếm đệ quy}
var
    i:integer;{Bien lap}
    k:integer;{Can tren pham vi lap cua tang nay}

```

```

function ok(num:integer):boolean;{Xet so num co the duoc cau tao tai thoi diem hien tai hay khong}
{De nang cao hieu suat, o day dung ket qua quy hoach dong}
var
    i,j:integer;{Bien lap}
begin
    if num<=range then begin{So num can xet nam trong [n/2]}
        ok:=(time[num]=2);{Dung truc tiep dieu kien so so hang toi thieu co bang 2 hay khong}
        exit;
    end;
    for i:=step-1 downto 1 do begin
        if a[i]+a[i]<num then break;
        if time[num-a[i]]=1 then begin
            {time[t]=1 cho biet t da xuat hien trong day hien co, tranh phai dung vong lap de xet}
            ok:=true;
            exit;
        end;
    end;
    ok:=false;
end;

```

```

procedure evltime;{Thu tuc con quy hoach dong}
var
    i,j:integer;{Bien lap}
    p:integer;{Can tren cua lan truy hoi quy hoach dong nay}
begin
    p:=k;
    if p>range then p:=range;
    for i:=a[step-1]+1 to p do begin
        time[i]:=time[i-a[step-1]]+1;{Gan gia tri dau cho ham muc tieu}
        for j:=step-2 downto 2 do{Liet ke quyet dinh}
            if time[i-a[j]]+1<time[i] then time[i]:=time[i-a[j]]+1;
        end;
    end;
end;

```

```

function h(num:integer):byte;{Ham uoc luong don gian ban dau h}
var
    i:integer;{Bien dem vong lap}
begin
    i:=0;
    while num<n do begin
        num:=num shl 1;
        inc(i);
    end;

```

```

end;
h:=i;
end;

function cut1:boolean;{Phan doan cat tia ban dau, goi truc tiep ham uoc luong h}
begin
  if h(a[step])+step>=best then cut1:=true else cut1:=false;
end;

function cut2:boolean;{Phan doan cat tia dung ham uoc luong cai tien}
var
  pt:integer;{Gia tri k(a[step]), tuc so mu cua 2 trong a[step]}
  rest:integer;{So trong day moi thoa k(rest)=k(n)}
  i:integer;{Bien dem vong lap}
begin
  if h(a[step]+a[step-1])+step+1<best then begin
    {Neu buoc dau cua day moi chon a[step]+a[step-1] thay vi 2*a[step] thi chac chan se bi cat tia;
    day la dieu kien can de ham uoc luong moi phat huy tac dung.
    }
    cut2:=false;
    exit;
  end;

  pt:=getk(a[step]);{Tinh k(a[step])}
  if pt>kn then rest:=a[step-1]
  else
    if pt=kn then rest:=a[step]
    else rest:=maxint;
    {Gan gia tri dau cho rest de phong khi moi so trong day moi co k nho hon k(n)}

  i:=0;
  repeat
    a[step+i+1]:=a[step+i] shl 1;
    inc(i);
    if pt+i=kn then rest:=a[step+i];
  until a[step+i]>n;
  dec(i);
  {Doan tren la qua trinh cau tao day}

  if (n-a[step+i]>rest)and(step+i+2>=best) then cut2:=true else cut2:=false;{Phan doan cat tia}
end;

begin{Find}
  if a[step-1]+a[step-1]>=n then begin
    {So lon nhat hien tai trong day da vuot qua [n/2], nen dung truc tiep ket qua quy hoach dong}
    if time[n-a[step-1]]+step-1<best then begin{Xac dinh co loi giai}
      best:=time[n-a[step-1]]+step-1;
      keep:=a;
    end;
  end;
end;

```

```

end;
  exit;
end;

k:=a[step-1]+a[step-1];{Tinh can tren pham vi lua chon cua a[step]}
evltime;{Goi thu tuc con quy hoach dong}

inc(a[0]);
for i:=k downto a[step-1]+1 do
  if ok(i) then begin
    if i<=range then time[i]:=1;
    a[step]:=i;
    if cut1 then break;{Goi phan doan cat tia 1}
    if cut2 then continue;{Goi phan doan cat tia 2}
    find(step+1);{Goi de quy tang tim kiem tiep theo}
  end;
  dec(a[0]);
end;

procedure formkeep;{Sinh day toi uu keep}
{Trong luc tim kiem, neu a[step]>[n/2] thi dung truc tiep ket qua quy hoach dong,
nen phan cuoi cua day ket qua toi uu thuc ra chua duoc tao ra. Qua trinh nay
bo sung phan do. 0 day van can de quy va quay lui, thuc chat la mot tim kiem
quy mo nho; tuy nhien, vi ket qua quy hoach dong bieu thi viec chon so it nhat
cac so tu day hien co de cong thanh n, nen cac so nay co the duoc sap thu tu.
}
var
  found:boolean;{Co danh dau da tim thay day toi uu}

procedure check(from,left:integer);{Quay lui sinh phan cuoi cua day toi uu}
  {from la can tren cua vong lap trong tang hien tai, dung de sap thu tu;
  left la phan con lai can thu duoc bang phap cong}
var
  i:integer;{Bien lap}
begin
  if keep[0]=best then begin
    if left=0 then found:=true;{Tim thay day toi uu}
    exit;
  end;
  inc(keep[0]);
  for i:=from downto 1 do{Liet ke cac so trong day}
    if keep[i]<=left then begin
      keep[keep[0]]:=keep[keep[0]-1]+keep[i];
      check(i,left-keep[i]);{Goi tang tim kiem tiep, nhung so duoc dung khong vuot qua keep[i]}
      if found then exit;
    end;
  dec(keep[0]);
end;

```

```

begin{FromKeep}
    found:=false;
    check(keep[0],n-keep[keep[0]]);{Goi qua trinh tim kiem}
end;

begin{Search}
    kn:=getk(n);{Tinh k(n)}
    time[0]:=0;
    time[1]:=1;
    a[0]:=1;
    a[1]:=1;
    range:=n div 2;
    {Doan tren khoi tao truoc khi tim kiem}

    find(2);{Goi qua trinh tim kiem}
    formkeep;{Sau khi tim kiem ket thuc, sinh day cau tao day du trong keep}
end;

function guess(n:integer):integer;{Tien xu ly uoc can bang de quy}
{Ghi chu:
    1. Sau khi chuong trinh con guess ket thuc, gia tri tra ve la ket qua uoc luong cho n;
        dong thoi, mang keep nhan duoc day tuong ung.
    2. De uoc luong cang chinh xac cang tot, guess xet dong thoi hai chien luoc phan tich,
        tuc la dung cau truc quay lui.
    3. Vi moi lan goi de quy guess deu phai ghi ket qua cuoi cung vao keep, moi lan guess
        chi thao tac mot phan cua mang keep va can den cac mang tam kp, kpt.
}
var
    num:integer;{Phan con lai sau khi tach het cac thua so 2 trong n}
    pfact:integer;{Bien bieu thi thua so nguyen to cua num}
    best:integer;{Ghi gia tri uoc luong tra ve khi goi xuong duoi}
    b2:integer;{Ghi so thua so 2 trong num}
    s:integer;{So buoc uoc luong cho n}
    i:integer;{Bien lap}
    sq:integer;{Phan nguyen can bac hai cua num, lam can tren khi phan tich thua so}
    s1,k2,k:integer;{Bien tam}
    kpt:atype;{Tam ghi day ket qua toi uu}

begin
    num:=n;
    b2:=0;
    while not odd(num) do begin
        num:=num div 2;
        inc(b2);
        inc(keep[0]);
        keep[keep[0]]:=power2[b2];
    end;
    s:=b2+1;

```

```

k2:=keep[0];
{Doan tren tach het cac thua so 2 trong n, phan con lai la num}

if num<=mp then begin{Neu num du nho, khong vuot qua mp, thi goi truc tiep qua trinh tim kiem}
  best:=maxint;
  search(num,best,kp);{Goi tim kiem cho n, day thu duoc luu trong kp}
  inc(s,best-1);
  for i:=2 to best do keep[i+keep[0]-1]:=kp[i];
  inc(keep[0],best-1);
end
else begin{Xu ly bang phuong phap uoc luong}
  k:=keep[0];

  best:=guess(num-1);{Goi de quy guess}
  keep[keep[0]+1]:=keep[keep[0]]+1;
  inc(keep[0]);
  inc(s,best);
  {Doan tren la chien luoc uoc luong thu nhat: tru 1 khoi num roi uoc luong num-1}

  sq:=trunc(sqrt(num));
  pfact:=3;
  while (pfact<=sq)and(num mod pfact>0) do inc(pfact,2);
  if pfact<=sq then begin
    kpt:=keep;
    keep[0]:=k2;
    s1:=b2+1;
    {Doan tren khoi tao truoc khi dung chien luoc thu hai}

    k:=keep[0];
    best:=guess(pfact);
    inc(s1,best-1);
    {Doan tren goi de quy guess de uoc luong thua so nguyen to pfact}

    k:=keep[0];
    best:=guess(num div pfact);
    for i:=k+1 to keep[0] do keep[i]:=pfact*keep[i];
    inc(s1,best-1);
    {Doan tren uoc luong [num/pfact]}

    if s1<s then s:=s1 else keep:=kpt;{So sanh hai chien luoc}
  end;
  {Doan tren la chien luoc uoc luong thu hai: tach mot thua so nguyen to nho cua num roi xu ly.
  Ket qua uoc luong duoc luu trong s1; kpt tam luu day do chien luoc thu nhat tao ra.
  }
end;

for i:=k2+1 to keep[0] do keep[i]:=keep[i]*power2[b2];

```

```

    guess:=s;{Tra ve ket qua uoc luong}
end;

```

```

procedure output;{Xuat ket qua}
var
    i,j,k:integer;{Bien lap}
begin
    rewrite(f);
    writeln(f,'X1');
    for i:=2 to best do begin
        for j:=1 to i-1 do begin
            for k:=j to i-1 do
                if keep[j]+keep[k]=keep[i] then break;
            if keep[j]+keep[k]=keep[i] then break;
        end;
        writeln(f,'X',i,'=X',j,'*X',k);
    end;
    writeln(f,'Times of multiplies=',best-1);
    close(f);
end;

```

```

begin{Chuong trinh chinh}
    init;{Doc du lieu va khoi tao}
    best:=guess(n);{Goi tien xu ly uoc can}
    search(n,best,keep);{Goi qua trinh tim kiem chinh}
    output;{Xuat ket qua}
end.

```